



THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE
LINGUAGEM PARA CORREÇÃO E FEEDBACK DE
EXERCÍCIOS DE POO**

LAVRAS – MG

2026

THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE LINGUAGEM PARA
CORREÇÃO E FEEDBACK DE EXERCÍCIOS DE POO**

Trabalho de Conclusão de Curso apresentado
à Universidade Federal de Lavras, como
parte das exigências do Curso de Ciência da
Computação, para a obtenção do título de
Bacharel

Prof. Dr. Julio César Alves
Orientador

**LAVRAS – MG
2026**

**Ficha catalográfica elaborada pela Coordenadoria de Processos Técnicos da Biblioteca
Universitária da UFLA, com dados informados pelo(a) próprio(a) autor(a).**

Martins, Thallys Henrique

Aplicação e Avaliação de Grandes Modelos de Linguagem
para Correção e Feedback de Exercícios de POO / Thallys Hen-
rique Martins. 6ª ed. rev., atual. e ampl. – Lavras : UFLA,
2026.

30p. : il.

Tese(doutorado)–Universidade Federal de Lavras, 2026.

Orientador: Prof. Dr. Julio César Alves .

Bibliografia.

1. Inteligência Artificial. 2. Educação. 3. POO. 4. Feedback
Automatizado. I. Universidade Federal de Lavras. II. Título.

CDD 808.066

THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE LINGUAGEM PARA
CORREÇÃO E FEEDBACK DE EXERCÍCIOS DE POO**

**APPLICATION AND EVALUATION OF LARGE LANGUAGE MODELS FOR
CORRECTION AND FEEDBACK OF OOP EXERCISES**

Trabalho de Conclusão de Curso apresentado
à Universidade Federal de Lavras, como
parte das exigências do Curso de Ciência da
Computação, para a obtenção do título de
Bacharel

APROVADA em Data de Defesa.

Prof. MSc. Antônio Banca Um	UFM
Prof. DSc. João Banca Dois	FCO
Profa. Esp. Eliza Banca Três	BELMIS
Prof. Esp. Zorbison Uberplax IV	UFO

Prof. Dr. Julio César Alves
Orientador

**LAVRAS – MG
2026**

Espaço reservado à epígrafe.

(Autor Desconhecido)

RESUMO

A avaliação de exercícios práticos na disciplina de Programação Orientada a Objetos (POO) é fundamental para o aprendizado, mas exige um alto esforço do docente, especialmente em relação à consistência e clareza do feedback. Ferramentas de correção automática (como o JavaCorrige) são valiosas, mas muitas vezes limitadas na qualidade do retorno fornecido ao estudante. Com o avanço das Large Language Models (LLMs), torna-se viável a criação de um sistema de correção que ofereça feedback personalizado e sofisticado, superando as limitações dos sistemas tradicionais. A abordagem adotada neste projeto será a utilização da biblioteca LangGraph para criar um sistema de correção desacoplado, garantindo a flexibilidade técnica para comparar e alternar o modelo de IA (LLM) utilizado (ex: Gemini, OpenAI), o que é vital para a longevidade e validade da pesquisa.

Palavras-chave: programação orientada a objetos; grandes modelos de linguagem; langgraph; feedback pedagógico; avaliação de código.

ABSTRACT

The evaluation of practical exercises in Object-Oriented Programming (OOP) is fundamental for learning but demands high effort from instructors, particularly concerning consistency and feedback clarity. Automated correction tools (such as JavaCorrige) are valuable yet often limited in the quality of the feedback provided to the student. With the advancement of Large Language Models (LLMs), it becomes feasible to create a correction system that offers personalized and sophisticated feedback, overcoming the limitations of traditional systems. The approach adopted in this project involves using the LangGraph library to create a decoupled correction system, ensuring technical flexibility to compare and switch the AI model (LLM) used (e.g., Gemini, OpenAI), which is vital for the longevity and validity of the research.

Keywords: object-oriented programming; large language models; langgraph; pedagogical feedback; code evaluation.

SUMÁRIO

1	INTRODUÇÃO	8
1.1	Objetivos	9
1.2	Estrutura do Texto	9
2	REFERENCIAL TEÓRICO	10
2.1	Ensino de Programação Orientada a Objetos (POO)	10
2.2	Correção Automática de Código	11
2.3	Modelos de Linguagem de Grande Escala (LLMs)	12
2.4	IA como Tutor e Orquestração Pedagógica	13
2.5	A Biblioteca Langgraph	13
2.6	Trabalhos Relacionados	13
3	METODOLOGIA	14
3.1	Contexto da Pesquisa	14
3.2	O Problema da “Correção Estrutural Binária”	14
3.3	Fundamentos do Feedback Formativo com LLMs	15
3.4	O uso de Java e Python no Ecossistema	15
3.5	Etapas Metodológicas do Desenvolvimento	16
3.5.1	Levantamento de Requisitos e Definição de Escopo	16
3.5.2	Modelagem Conceitual: Eixos de Engenharia e Pedagogia	16
3.5.3	Modelagem Funcional: Núcleos de Componentes do Sistema	17
4	DESENVOLVIMENTO DO SISTEMA	18
4.1	Arquitetura e Fluxo de Execução com <i>LangGraph</i>	18
4.2	Interface Gráfica e Gestão de Submissões	18
4.3	Módulo de Pré-processamento e Limpeza de Código	19
4.4	Engenharia de <i>Prompt</i> e Saída Estruturada	19
4.4.1	Diretrizes de Comportamento e Restrição de Escopo	20
4.5	Resiliência e Persistência de Dados	20
5	RESULTADOS E DISCUSSÃO	21
5.1	Análise dos Estudos de Caso: O Papel do Gabarito	21
5.2	Análise Qualitativa e Limitações Identificadas	21
5.2.1	Consistência Pedagógica e Clean Code	22
5.2.2	Limitações Técnicas e Rigor Geométrico	22

5.3	Evolução do Feedback: Segunda Estratégia de Prompt	22
	APÊNDICES	24

1 INTRODUÇÃO

A crescente demanda por profissionais qualificados na área de tecnologia tem impulsionado a busca por métodos de ensino de programação mais escaláveis e eficientes. No contexto do ensino superior, as disciplinas de Programação Orientada a Objetos (POO) representam um marco na formação do estudante, exigindo a transição do pensamento puramente algorítmico para uma visão baseada em design, abstração e arquitetura de sistemas. No entanto, a avaliação qualitativa de exercícios nesse paradigma apresenta desafios significativos.

O processo de fornecer feedback detalhado e personalizado é uma tarefa que consome um volume considerável de tempo do docente. De acordo com Pereira e Mello (2025), a falta de um retorno pedagógico imediato e consistente pode prejudicar o ciclo de aprendizagem do discente, especialmente quando os erros cometidos não são apenas sintáticos, mas conceituais, envolvendo princípios fundamentais do paradigma, como encapsulamento e coesão (Cipriano; Alves, 2023).

Ferramentas de correção automática tradicionais, embora úteis para validar se um código compila ou passa em testes unitários, frequentemente falham em oferecer uma análise qualitativa profunda. Estas ferramentas costumam atuar como validadores estruturais, focando no “que” o código faz e negligenciando o “como” ele foi projetado. Com a recente popularização dos Grandes Modelos de Linguagem (LLMs), abre-se uma nova fronteira para a automação pedagógica, permitindo a geração de feedbacks que se assemelham à análise realizada por um tutor humano.

Entretanto, a integração direta de LLMs em sistemas de ensino esbarra em obstáculos técnicos, como a tendência desses modelos a gerar informações imprecisas (alucinações) e a necessidade crítica de supervisão humana para garantir a sustentabilidade pedagógica (Pereira; Mello, 2025). Além disso, a rápida evolução tecnológica torna arriscado o desenvolvimento de sistemas dependentes de um único provedor de inteligência artificial.

Este trabalho propõe a aplicação e avaliação de um sistema de feedback automatizado construído sobre uma arquitetura desacoplada. Utilizando o framework LangGraph para orquestração de estados e o modelo Gemini 2.5 Flash via SDK oficial. A pesquisa foca na capacidade de alternar entre diferentes modelos generativos, garantindo a longevidade da ferramenta e avaliando como a presença de uma solução de referência (gabarito) altera a profundidade qualitativa do feedback fornecido ao estudante de POO.

1.1 Objetivos

O objetivo deste trabalho é desenvolver e avaliar um protótipo de sistema de correção de código para exercícios de POO utilizando a biblioteca LangGraph, com foco na garantia de flexibilidade técnica e na qualidade do feedback gerado por IA. Para isso, os seguintes objetivos específicos foram estabelecidos:

- Desenvolver o fluxo principal de correção e análise utilizando a biblioteca LangGraph, garantindo a flexibilidade técnica para alternar e comparar diferentes Modelos de Linguagem Grande (LLMs).
- Automatizar a geração de anotações estruturais e gabaritos de correção do JavaCorrige por meio da IA, reduzindo o esforço do docente.
- Implementar um módulo de IA para enriquecer o relatório de resultados, transformando notas quantitativas em feedback qualitativo e claro para os estudantes.
- Utilizar a IA para realizar a correção propriamente dita do código, e integrar seus resultados com a correção estrutural do JavaCorrige para compor uma avaliação final mais completa.
- Validar a eficácia, precisão e consistência das correções e dos feedbacks gerados pela IA em um conjunto de exercícios reais de Programação Orientada a Objetos.
- Analisar e documentar os custos, as limitações e as licenças das LLMs disponíveis para garantir a viabilidade econômica do sistema a longo prazo.

1.2 Estrutura do Texto

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta o referencial teórico que fundamenta a pesquisa realizada. Nele, são discutidos os principais conceitos, teorias e trabalhos relacionados ao tema abordado, proporcionando uma base sólida para a compreensão do problema de pesquisa e a justificativa da metodologia adotada. A revisão da literatura é organizada de forma a destacar as contribuições mais relevantes e identificar lacunas que a presente pesquisa busca preencher.

2.1 Ensino de Programação Orientada a Objetos (POO)

O ensino de Programação Orientada a Objetos (POO) configura-se como um dos maiores desafios nos currículos de Ciência da Computação. Diferente do paradigma estruturado, que foca em sequências de instruções e lógica procedimental, a POO exige que o discente desenvolva uma capacidade de abstração para modelar entidades do mundo real em componentes de software interconectados.

De acordo com Booch (2007), o modelo de objetos fundamenta-se em quatro elementos essenciais: abstração, encapsulamento, modularidade e hierarquia. O autor defende que o domínio desses conceitos é o que permite gerenciar a complexidade inerente ao desenvolvimento de software moderno, enfatizando que a orientação a objetos transcende a sintaxe das linguagens de programação e deve ser compreendida como um paradigma de design e organização de sistemas.

A transição para o paradigma orientado a objetos impõe um desafio cognitivo significativo, visto que estudantes iniciantes frequentemente enfrentam barreiras para consolidar o “pensar em objetos”. Segundo Eckerdal e Berglund (2005), o aprendizado da POO exige que o aluno ultrapasse a compreensão meramente sintática da linguagem para atingir um nível de abstração que permita a modelagem de problemas complexos.

Complementarmente, Armstrong (2006) argumenta que essa transição é dificultada quando o ensino foca excessivamente na implementação em detrimento dos conceitos fundamentais — os “quarks” do desenvolvimento. Para a autora, a plena compreensão do paradigma exige o domínio de conceitos de estrutura (como objetos, classes, herança e encapsulamento) e de comportamento (como polimorfismo e passagem de mensagens). Sem essa base conceitual, o aluno tende a replicar vícios da lógica procedimental, utilizando classes apenas como depósitos de

dados e manipulando atributos de forma externa. Ao falhar em delegar responsabilidades aos objetos através de métodos e troca de mensagens, o estudante produz sistemas que, embora funcionalmente corretos, carecem das vantagens de manutenibilidade e reuso típicas de arquiteturas orientadas a objetos.

Diferente de disciplinas introdutórias, que frequentemente recorrem a ferramentas de correção automática baseadas em testes de saída — a exemplo do Dredd (SCHNEIDER, 2021), desenvolvido e amplamente utilizado no contexto da Universidade Federal de Lavras —, a disciplina de POO exige uma análise que transcenda o veredito de “sucesso” ou “falha” na execução. Embora sistemas de avaliação automática sejam eficazes para validar a lógica de algoritmos lineares, eles apresentam limitações pedagógicas ao não contemplar a semântica e os princípios de design do código produzido pelo estudante, o que pode mascarar lacunas na compreensão do paradigma de objetos.

2.2 Correção Automática de Código

Historicamente, o *feedback* automatizado em ambientes educacionais tem se concentrado em aspectos quantitativos, como a correção de respostas objetivas ou a validação de código por meio de testes unitários. Embora essas abordagens garantam agilidade, elas limitam-se a informar se um resultado está correto ou incorreto, falhando em fornecer a orientação pedagógica necessária para que o discente compreenda a origem do seu erro conceitual.

Essa limitação é evidenciada por Keuning et al. (2018), que, ao realizarem uma revisão sistemática de 101 ferramentas de avaliação automática, constataram que a grande maioria foca estritamente na identificação de erros. Em contrapartida, são escassos os sistemas que oferecem orientações sobre como o aluno deve proceder ou que fornecem explicações sobre os conceitos teóricos envolvidos. Esse cenário indica uma falta de suporte ao processo de aprendizagem e à autonomia do aluno. Isso ocorre porque as ferramentas focam no resultado imediato do exercício, em vez de priorizar a formação de uma base teórica sólida.

Com o advento das *LLMs*, torna-se possível realizar uma análise semântica e qualitativa que atinja um nível de *feedback* que se aproxima do oferecido por um tutor humano. A viabilidade técnica dessa evolução é validada por estudos como o de Wang (2023), que demonstrou ser possível automatizar o *feedback* docente com alta fidelidade ao utilizar modelos especializados. A arquitetura proposta alcançou uma pontuação METEOR (Metric for Evaluation of Transla-

tion with Explicit Ordering) superior a 96%, o que indica que as respostas geradas são quase indistinguíveis do julgamento de um professor especializado. O uso da métrica METEOR é fundamental para essa conclusão, pois ela avalia a qualidade do texto considerando sinônimos e a coesão lógica, garantindo que o sistema ofereça um suporte pedagógico robusto e confiável para o aprendizado de programação.

2.3 Modelos de Linguagem de Grande Escala (LLMs)

Os Modelos de Linguagem de Grande Escala, conhecidos pela sigla *LLM* (*Large Language Models*), são sistemas de inteligência artificial baseados em redes neurais profundas, projetados para compreender, gerar e manipular textos em linguagem natural ou código de programação. O termo “Grande Escala” refere-se tanto ao imenso volume de dados utilizados no treinamento quanto ao número de parâmetros da rede — chegando a centenas de bilhões —, o que permite ao modelo capturar nuances complexas de contexto, semântica e sintaxe.

A base tecnológica das *LLMs* modernas é a arquitetura *Transformer*, introduzida por Vaswani et al. (2017). O diferencial dessa arquitetura é o mecanismo de atenção (*attention*), que permite ao modelo processar todas as partes de uma sequência de texto simultaneamente. Diferente de modelos anteriores que liam o texto de forma linear, o *Transformer* consegue identificar quais palavras ou símbolos são mais relevantes entre si, independentemente da distância no texto. Em programação, essa capacidade é vital, pois permite que a IA compreenda como uma declaração de classe na linha inicial influencia um método implementado centenas de linhas abaixo.

De acordo com Bommasani et al. (2021), esses sistemas são classificados como “modelos de fundação”, pois são treinados em bases de dados massivas e diversificadas, adquirindo uma capacidade de generalização que permite a execução de tarefas para as quais não foram explicitamente programados. No ensino de POO, isso significa que a *LLM* não apenas identifica erros de compilação, mas é capaz de interpretar a intenção do estudante, avaliar a elegância do *design* e sugerir refatorações baseadas em boas práticas de mercado.

Dessa forma, as *LLMs* superam as limitações das ferramentas tradicionais de correção ao oferecerem uma análise qualitativa. Elas deixam de atuar como simples validadores de “sucesso ou falha” para funcionarem como motores de inferência semântica, capazes de sustentar um diálogo pedagógico fluido e contextualizado.

2.4 IA como Tutor e Orquestração Pedagógica

De acordo com Pereira e Mello (2025), o uso de métodos manuais de avaliação em cursos de larga escala frequentemente resulta em atrasos na correção, o que impacta negativamente a motivação e a participação dos discentes. Os autores defendem que as LLMs oferecem uma solução para este problema ao possibilitarem respostas em tempo real, garantindo que o retorno pedagógico seja imediato e relevante para o processo de aprendizagem.

No entanto, um dos maiores desafios pedagógicos reside em evitar que a IA forneça a resposta pronta, comprometendo o esforço cognitivo do aluno. Jacobs e Jaschke (2023) demonstram que, ao utilizar modelos avançados como o GPT-4, é possível mitigar quase integralmente essa limitação, fornecendo sugestões e orientações sem expor a solução final. Os autores ressaltam que, para manter essa eficácia, é vital evitar o envio de informações contraditórias à IA (como conflitos entre o código do aluno e o gabarito), o que reforça a necessidade de fluxos de dados controlados por orquestradores para garantir a integridade do papel de tutor.

2.5 A Biblioteca Langgraph

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2.6 Trabalhos Relacionados

3 METODOLOGIA

Neste capítulo, descrevem-se os procedimentos, técnicas e materiais planejados para o desenvolvimento e validação do sistema de correção automatizada. O objetivo é detalhar o percurso metodológico e as decisões técnicas que fundamentarão a construção do protótipo, garantindo a transparência e a replicabilidade dos experimentos.

3.1 Contexto da Pesquisa

A pesquisa classifica-se como um estudo aplicado de natureza tecnológica. O contexto da investigação reside nas disciplinas GAC106 e GAC125, que tratam de conceitos e práticas de Programação Orientada a Objetos, do Departamento de Computação Aplicada (DAC-ICET) da Universidade Federal de Lavras (UFLA). Pretende-se utilizar submissões de códigos reais de discentes, anonimizadas, como base para os experimentos, visando a validação da ferramenta em um cenário acadêmico autêntico. A escolha por este ambiente educacional específico justifica-se pela necessidade de avaliar a eficácia do sistema em um contexto onde a diversidade de soluções e a complexidade dos exercícios são representativas dos desafios enfrentados pelos estudantes.

3.2 O Problema da “Correção Estrutural Binária”

Tradicionalmente, sistemas de análise automática de código, como o *Dredd* ou *frameworks* de testes unitários (como o *JUnit*), operam sob uma lógica de validação funcional baseada em casos de teste. Nesses modelos, cada caso de teste individual possui um veredito binário, verificando estritamente se a saída gerada pelo programa corresponde ao valor esperado para uma determinada entrada. Embora o resultado final de um exercício possa ser composto pelo sucesso em múltiplos testes, refletindo uma nota proporcional, a avaliação restringe-se ao comportamento externo do programa.

Embora eficazes para validar a funcionalidade, tais abordagens tratam apenas da validação de entrada e saída do programa e falham em identificar problemas na forma de construção do programa (muito importantes em POO, como a falta de encapsulamento, por exemplo) ou em oferecer pistas pedagógicas que auxiliem o estudante na construção do conhecimento. A

partir dessa percepção, o presente projeto não busca apenas automatizar a nota, mas sim simular a análise de um instrutor humano.

Visando solucionar a limitação das avaliações baseadas estritamente em resultados de execução, estabelece-se a proposta central deste sistema: utilizar Grandes Modelos de Linguagem (LLMs) orquestrados por grafos para realizar uma análise semântica profunda. Essa abordagem permite confrontar o código-fonte do discente com o *design* estrutural esperado pelo professor, gerando um retorno rico em contexto técnico que transcende a mera verificação de casos de teste.

3.3 Fundamentos do Feedback Formativo com LLMs

Nesta seção, justifica-se a escolha da Inteligência Artificial não apenas como uma ferramenta de automação, mas como um agente de Feedback Formativo. Diferente da correção somativa (que apenas atribui uma nota), o feedback formativo foca no processo de aprendizagem.

Pretende-se utilizar as capacidades de raciocínio das LLMs (Large Language Models) para simular o comportamento de um tutor humano. O sistema deverá ser configurável para atuar em dois níveis de intervenção:

- **Modo de Scaffolding (Andaime):** Onde a IA identifica o estágio de compreensão do aluno e fornece “pistas” em vez de entregar a resposta pronta, incentivando a reflexão sobre conceitos de POO.
- **Modo Resolutivo:** Onde a IA fornece a sugestão de correção completa, entregando o código Java corrigido para que o aluno compare com sua tentativa inicial e identifique as divergências técnicas de forma direta.

3.4 O uso de Java e Python no Ecossistema

A arquitetura do projeto será fundamentada na interoperabilidade entre duas linguagens distintas. O Java foi selecionado como a linguagem de estudo dos discentes, por ser o padrão das disciplina de POO na UFLA e por seu rigor sintático.

Em contrapartida, o motor de análise (*back-end*) será desenvolvido em *Python*, utilizando a biblioteca *LangGraph*. A escolha por *Python* é motivada pela maturidade de seu ecossistema para Inteligência Artificial e pela facilidade de manipulação de *prompts*, permitindo uma integração fluída com *APIs* de Grandes Modelos de Linguagem (*LLMs*).

3.5 Etapas Metodológicas do Desenvolvimento

O desenvolvimento do sistema será conduzido de forma iterativa, seguindo as fases detalhadas a seguir:

3.5.1 Levantamento de Requisitos e Definição de Escopo

A primeira etapa do desenvolvimento consistirá no levantamento de requisitos, com o objetivo de definir o escopo, a estrutura e o propósito do sistema de correção automatizada. A partir disso, planeja-se a criação de uma ferramenta que processe submissões de código Java e forneça retornos claros e instrutivos.

No aspecto pedagógico, os conteúdos a serem avaliados pelo sistema serão definidos com base nas ementas das disciplinas GAC106 e GAC125, que compõem o núcleo de Programação Orientada a Objetos da UFLA, contemplando inicialmente os conceitos de: Classes e Objetos, Atributos e Métodos, Encapsulamento, Construtores, Herança e Polimorfismo. A escolha desses tópicos justifica-se por serem os pilares fundamentais da disciplina.

Durante a concepção do projeto, também serão analisadas outras soluções educacionais já existentes que ilustram diferentes abordagens para a correção de código. Essa análise servirá para identificar as melhores práticas e as limitações das ferramentas atuais.

Reconhecidas as necessidades para o sistema — a partir de testes preliminares de viabilidade e do *brainstorm* de ideias —, serão definidas as funcionalidades essenciais.

3.5.2 Modelagem Conceitual: Eixos de Engenharia e Pedagogia

- **Eixo de Engenharia:** Foca na robustez técnica, garantindo que o fluxo de dados entre o código Java do aluno e o motor em *Python* seja eficiente. Este eixo abrange a orquestração

de estados via *LangGraph*, e a garantia de que a saída da *LLM* seja processável (formato *JSON*).

- **Eixo Pedagógico:** Concentra-se na qualidade do ensino. Define-se aqui como o sistema deve se comportar para ser um tutor efetivo, estabelecendo o tom de voz do *feedback* e garantindo que os conceitos de POO sejam abordados com precisão técnica, seja no modo resolutivo ou de scaffolding.

3.5.3 Modelagem Funcional: Núcleos de Componentes do Sistema

- **Núcleo de Infraestrutura (Lógico):** Responsável por interpretar o código do aluno, identificar os conceitos de POO presentes e compará-los com o design esperado pelo professor.
- **Núcleo de Inteligência (Pedagógico):** Encapsula a lógica de construção do *feedback*, garantindo que as respostas da *LLM* sejam formatadas de maneira clara e instrutiva, seguindo o *schema JSON* definido.
- **Núcleo de Normalização (Híbrido):** Atua no pré-processamento das submissões. Será incorporado rotinas de limpeza via *Regex* e formatação automatizada para garantir que a IA receba um código padronizado, reduzindo o risco de interpretações errôneas causadas por variações estéticas do texto.

4 DESENVOLVIMENTO DO SISTEMA

Neste capítulo, detalha-se a implementação técnica do protótipo de correção automatizada. A solução foi estruturada para garantir a interoperabilidade entre as linguagens Java e Python, utilizando orquestração de estados para simular o comportamento de um tutor pedagógico.

4.1 Arquitetura e Fluxo de Execução com *LangGraph*

O núcleo do sistema baseia-se em uma arquitetura de grafos de estado utilizando a biblioteca *LangGraph*. O fluxo de trabalho (*workflow*) foi projetado como um *StateGraph* cíclico, onde o estado da correção (*CorrectionState*) é transportado entre os nós.

A implementação utiliza um nó principal denominado `correction_node`, responsável por consolidar o enunciado, o gabarito e o código do discente. O sistema suporta dois modos de operação:

- **Modo Único:** Avaliação de um projeto individual selecionado.
- **Modo Lote (*Batch*):** Processamento sequencial de múltiplas pastas de alunos, permitindo a escala da correção para turmas inteiras.

4.2 Interface Gráfica e Gestão de Submissões

Para facilitar o uso docente, desenvolveu-se uma interface gráfica utilizando a biblioteca *Tkinter*, estruturada em componentes de cartões de seleção (*CardSelecao*). A ferramenta permite a gestão flexível de diferentes formatos de entrada:

- **Processamento de Arquivos Compactados:** O sistema identifica e extrai automaticamente arquivos `.zip`, mapeando os diretórios temporários de volta à pasta original para garantir que os relatórios gerados sejam salvos no local correto.
- **Seleção Dinâmica de Conceitos:** Através de um seletor baseado em arquivos *Markdown*, o professor pode filtrar quais tópicos de POO devem ser priorizados na avaliação atual.

4.3 Módulo de Pré-processamento e Limpeza de Código

Uma etapa crítica para a precisão da análise e economia de *tokens* é a higienização do código-fonte. Diferente de uma limpeza simples por *Regex*, o sistema implementa uma função robusta denominada `remove_java_comments`.

Essa função realiza uma análise léxica básica para garantir que sequências de caracteres como `/*` ou `//` não sejam removidas caso estejam contidas dentro de *strings* ou literais de caractere. A lógica opera em:

1. **Preservação de Literais:** Identificação de aspas duplas e simples para evitar remoções indevidas.
2. **Limpeza de Comentários:** Remoção de comentários de linha simples e de blocos (*multi-line*), incluindo suporte a comentários aninhados.
3. **Formatação Estética:** O código pode ser opcionalmente processado pelo *google-java-format* via JAR externo, garantindo um padrão visual consistente antes da análise pela LLM.

4.4 Engenharia de *Prompt* e Saída Estruturada

Para assegurar que o *feedback* seja tecnicamente preciso e pedagogicamente útil, implementou-se uma estrutura de saída rígida baseada em *JSON*. O *prompt* impõe uma persona de docente e atua como o *Gatekeeper* do sistema.

Tabela 4.1 – Campos do *Schema* JSON e Intenção Pedagógica

Campo do JSON	Descrição Técnica e Pedagógica
<code>avaliacao</code>	Status categórico: "Certo", "Errado" ou "Parcialmente Certo".
<code>justificativa</code>	Explicação detalhada dos erros e acertos sob a ótica de POO.
<code>dica_correcao</code>	Orientações processuais sem a revelação direta do código.
<code>sugestao_correcao</code>	Bloco de código Java corrigido para análise comparativa do aluno.

4.4.1 Diretrizes de Comportamento e Restrição de Escopo

O *prompt* incorpora regras rígidas para mitigar falhas comuns em corretores automatizados. Dentre as diretrizes implementadas, destacam-se:

- **Tolerância ao Estilo:** O modelo é instruído a tratar o gabarito do professor apenas como um exemplo de solução, não penalizando variações criativas ou escolhas estéticas que não impactem a segurança ou os requisitos do enunciado.
- **Segurança de Dados:** Implementou-se uma salvaguarda contra Dados Sensíveis (PII). O modelo deve identificar a presença de chaves ou senhas, sinalizando o risco na justificativa sem reproduzir o conteúdo sensível na saída.
- **Consciência de Pré-processamento:** O *prompt* informa à IA que o código foi limpo via *Regex*. Assim, o modelo é instruído a não penalizar a ausência de comentários solicitados no enunciado, focando exclusivamente na semântica dos métodos e atributos.
- **Delimitação Conceitual:** Caso a lista `{{CONCEITOS_A_AVALIAR}}` seja fornecida, a IA deve ignorar e não comentar sobre erros que fujam do escopo selecionado, garantindo um *feedback* adequado ao nível de conhecimento atual do discente.

4.5 Resiliência e Persistência de Dados

Considerando as limitações de taxa das APIs (*Rate Limits*), o sistema implementa uma lógica de *Exponential Backoff* com sorteio aleatório (*jitter*), permitindo até 5 tentativas em caso de erro `RESOURCE_EXHAUSTED`.

Ao final de cada ciclo, o sistema gera dois subprodutos:

1. **Relatório HTML:** Um documento visualmente rico aberto automaticamente no navegador.
2. **Persistência Local:** Uma cópia em formato JSON na pasta de respostas recebidas e opcionalmente uma cópia do relatório HTML diretamente na pasta de origem do discente, facilitando a devolução do *feedback*.

5 RESULTADOS E DISCUSSÃO

Este capítulo apresenta a análise dos experimentos realizados com o protótipo, avaliando o impacto da solução de referência (gabarito) na eficácia do *feedback* gerado. A análise foi estruturada em duas fases: testes controlados (Estudos de Caso I e II), utilizando a configuração inicial de *prompt*, e uma análise qualitativa de submissões reais que fundamentou o refinamento da estratégia pedagógica da IA.

5.1 Análise dos Estudos de Caso: O Papel do Gabarito

A experimentação inicial com os exercícios “Nave” e “Casa” utilizou a Primeira Estratégia de Prompt (Apêndice A) e permitiu observar como a ancoragem em um gabarito alterou a natureza da correção. Notou-se que, na ausência de uma solução de referência, a IA tendeu a atuar como um analisador funcional autônomo. Nesse estado, o modelo focou primordialmente na existência técnica dos componentes solicitados no enunciado e na identificação de erros de compilação.

Em contrapartida, a introdução do gabarito elevou o nível da avaliação. No cenário do exercício “Nave”, a presença da solução de referência permitiu que a IA identificasse não apenas a funcionalidade, mas a **equivalência técnica**. Por exemplo, o sistema pôde inferir que um código era idêntico à versão inicial não modificada, algo que uma análise sem gabarito interpretou apenas como ausência de implementação.

No exercício “Casa”, observou-se uma sofisticação maior no controle de estado (uso do booleano `porDoSolRealizado`). Com o suporte do gabarito, a IA demonstrou flexibilidade pedagógica ao classificar divergências paramétricas triviais — como pequenas variações de coordenadas — como aceitáveis, mantendo o foco na lógica de orquestração de eventos. Conforme discutido no referencial teórico, o foco residiu na arquitetura e não apenas na sintaxe, o que validou a abordagem híbrida proposta.

5.2 Análise Qualitativa e Limitações Identificadas

Após a validação controlada, o sistema processou códigos reais de discentes, revelando nuances críticas sobre o comportamento do modelo Gemini sob a primeira estratégia de *prompt*.

5.2.1 Consistência Pedagógica e Clean Code

Um dos resultados mais significativos foi a capacidade da IA de aplicar conceitos de *Clean Code* e coesão. Observou-se que o modelo identificou a “mistura de responsabilidades”, como o caso recorrente de alunos que implementaram a movimentação de um objeto dentro de métodos de controle ambiental. Além disso, a IA demonstrou rigor sobre o encapsulamento, sugerindo a remoção de métodos *getters* desnecessários que não foram solicitados, reforçando o conceito de interface mínima em POO.

Todavia, emergiu o desafio da “lógica no construtor”. A IA identificou corretamente como má prática a chamada de métodos públicos dentro do construtor, reforçando que este deve ser restrito à inicialização de estado. Essa descoberta convergiu com as melhores práticas da indústria mencionadas na fundamentação deste trabalho.

5.2.2 Limitações Técnicas e Rigor Geométrico

A análise qualitativa também revelou divergências indesejadas que comprometeram a experiência do usuário. Identificou-se uma **vulnerabilidade à indentação**, na qual o modelo alucinou erros de sintaxe (como classes não fechadas) devido a chaves desalinhadas no código do aluno. Este dado demonstrou que o pré-processamento via *linter* é indispensável para garantir a previsibilidade do sistema.

Outro ponto crítico foi o denominado “Rigor Geométrico”. A IA tendeu a penalizar divergências estéticas (cores e tamanhos) que não eram restritas pelo enunciado. Esse comportamento evidenciou a necessidade de refinamento nas instruções de sistema, visando ensinar o modelo a diferenciar erros de arquitetura graves de escolhas criativas irrelevantes.

5.3 Evolução do Feedback: Segunda Estratégia de Prompt

Diante das limitações técnicas e do rigor excessivo observados na fase inicial, foi desenvolvida a Segunda Estratégia de Prompt (Apêndice B). Esta versão incorporou instruções explícitas de tolerância estética e reforçou a priorização de conceitos estruturais de POO em detrimento de detalhes cosméticos. Para isso, o *prompt* foi fundamentado nos padrões de revisão

de código do *GitHub Awesome Copilot*, permitindo que o sistema avaliasse a qualidade técnica e a arquitetura de software seguindo critérios reais da indústria.

Os testes com a estratégia refinada demonstraram uma redução significativa de falsos positivos e a mitigação do “Rigor Geométrico” identificado anteriormente. Ao instruir o modelo a ignorar variações visuais não essenciais — como nomenclaturas alternativas de atributos e divergências de coordenadas que não ferem o enunciado —, o *feedback* tornou-se mais encorajador e focado em orientações processuais para a resolução de problemas.

APÊNDICES

APÊNDICE A – PROMPT DE CORREÇÃO UTILIZADO

Abaixo apresenta-se a estrutura exata das System Instructions utilizadas para orquestrar o comportamento da IA durante o processo de correção:

Listing 1 – Instrução de Sistema para Correção Automatizada

```
Você é um Professor de Programação Orientada a Objetos (POO) da UFLA
. Avalie o código Java do aluno considerando o enunciado. Forneça
feedback construtivo e educativo, focado em princípios de POO (
Encapsulamento, Herança, Lógica).
Sua resposta DEVE ser um JSON válido, SEM TEXTO ANTES OU DEPOIS, e
seguir EXATAMENTE este schema:
{
  "avaliacao": "Certo" | "Errado" | "Parcialmente Certo",
  "justificativa": string,
  "dica_correcao": string,
  "sugestao_correcao": string
}
No campo 'sugestao_correcao', coloque o código Java corrigido e
inclua comentários explicativos sempre que possível para ajudar o
aluno a entender as correções e boas práticas. Não inclua mensagens
de parabéns ou explicações fora do código. Se a sugestão envolver
mais de um arquivo, separe cada arquivo usando exatamente o padrão:
// --- ARQUIVO INÍCIO: NomeDoArquivo.java ---
(código do arquivo, com comentários explicativos se necessário)
// --- ARQUIVO FIM: NomeDoArquivo.java ---
No campo 'dica_correcao', coloque as orientações e dicas para o
aluno corrigir o código, sem incluir o código em si. Use ```java
apenas no prompt, não na resposta JSON. Responda integralmente em
português. Não adicione explicações fora do JSON.
```

Quando uma solução de referência (gabarito) é fornecida ao sistema, a instrução abaixo é concatenada ao prompt para garantir a ancoragem técnica:

Listing 2 – Instrução Adicional para Ancoragem por Gabarito

Avalie o código do aluno COMPARANDO DIRETAMENTE com o gabarito acima . Considere o gabarito como a única solução correta. Aponte diferenças, similaridades e se o código do aluno está igual, melhor ou pior que o gabarito. Siga a estrutura rígida definida no System Instruction.

APÊNDICE B – NOVA VERSÃO DO PROMPT DE CORREÇÃO UTILIZADO

Abaixo apresenta-se a estrutura exata das System Instructions utilizadas para orquestrar o comportamento da IA durante o processo de correção (nova versão):

Listing 3 – Prompt do Persona para Avaliação Automatizada

```
Você é um instrutor sênior de engenharia de software com mais de 15
anos de experiência em Java e design orientado a objetos. Adote um
tom claro, construtivo e pedagógico.
```

Listing 4 – Prompt do Sistema para Avaliação Automatizada

```
Siga estas regras rígidas ao avaliar código de aluno.

## Regras Rígidas

### JSON obrigatório

Sua resposta deve ser estritamente um JSON válido que satisfaça o
schema requerido e nada mais - sem texto livre antes ou depois.

Exemplo de shape (use exatamente estas chaves):
{
  "avaliacao": "Certo" | "Errado" | "Parcialmente Certo",
  "justificativa": "",
  "dica_correcao": "",
  "sugestao_correcao": ""
}

### Campos (regras específicas)

- sugestao_correcao: deve conter APENAS código Java corrigido - sem
comentários explicativos ou texto adicional. Para múltiplos arquivos
, separe exatamente assim (apenas conteúdo):

// --- ARQUIVO INÍCIO: NomeDoArquivo.java ---
<conteúdo do arquivo>
```

```
// --- ARQUIVO FIM: NomeDoArquivo.java ---
```

- justificativa: descrição técnica clara do porquê da avaliação (vinculada ao enunciado). Seja conciso e foque nos pontos relevantes.
- dica_correcao: instruções pedagógicas para o aluno corrigir (SEM incluir código).

```
### Erros e dados sensíveis
```

- Se detectar dados sensíveis (chaves, senhas, PII) no código, NAO reproduza o conteúdo sensível. Em justificativa indique sucintamente a presença e o impacto, sem expor os dados.

```
### Regras de comportamento do modelo
```

- Assuma que o código recebido já foi pré-processado: comentários removidos e formatado com google-java-format quando possível.
- Observação: se o enunciado exigir que o aluno adicione comentários, NAO penalize a ausência desses comentários, pois o pré-processamento os remove. Em vez disso, avalie a presença e o comportamento dos métodos esperados com base na semântica e nos efeitos observáveis do código.
- NAO inclua qualquer texto fora do JSON. Se for impossível produzir o JSON solicitado, retorne:

```
{ "avaliacao": "Erro", "justificativa": "<motivo>", "dica_correcao": "", "sugestao_correcao": "" }
```

- Se for fornecida a lista {{CONCEITOS_A_AVALIAR}}, avalie somente esses conceitos e ignore comentários sobre outros.

```
## Regra especial sobre gabarito e saída textual
```

- O gabarito do professor é um EXEMPLO de solução, não um molde obrigatório de formatação. NÃO penalize diferenças de formato/estilo quando a solução estiver correta em comportamento/semântica.
- Compare explícita e estritamente a saída textual (ex.: `System.out.println`) somente quando o enunciado exigir saída textual exata. Caso contrário, priorize corretude, tratamento de casos e aderência ao enunciado.
- Na justificativa, avalie se a solução demonstra entendimento do enunciado (trata casos, respeita restrições, implementa a lógica pedida). NÃO avalie nem penalize decisões criativas de formatação, posicionamento visual, nomes de variáveis ou escolhas estéticas quando elas NÃO impactarem a correção, segurança ou os requisitos do enunciado. Comente sobre diferenças apenas quando elas violarem restrições explícitas do enunciado ou influenciarem comportamento/saída do programa.

Linguagem e estilo

- Responda em Português (pt-BR).
- Use termos objetivos e técnicos; evite digressões.

Prioridades de correção

1. Correção/semântica (o programa atende o enunciado?)
2. Segurança e robustez
3. Legibilidade e estilo

Exemplos e verificações rápidas

- NÃO inclua trechos de código com informações sensíveis; sinalize a presença sem expor o conteúdo.

Princípios de revisão

- Seja específico: indique arquivos exatos e forneça exemplos concretos.
- Forneça contexto: explique POR QUE algo é um problema e seu impacto.
- Sugira soluções: mostre código corrigido quando aplicável.
- Seja construtivo: foque em melhorar o código, não em criticar o autor.
- Reconheça boas práticas quando presentes.
- Seja pragmático: nem toda sugestão precisa ser implementada imediatamente.
- Agrupar comentários correlatos: Evite múltiplos comentários sobre o mesmo tópico ou erro repetitivo.

Incorpore esses princípios ao preencher justificativa e dica_correcao - a justificativa deve conter a referência precisa e o impacto; a dica_correcao deve resumir próximos passos pedagógicos.

Siga estritamente estas regras; qualquer desvio deve resultar em avaliacao: "Erro" com justificativa explicando o problema.

